



How do I implement an interrupt service routine on Raspberry Pi?

Asked 4 years, 2 months ago Modified 1 year ago Viewed 7k times



4



There are several libraries like WiringPi, RPi and pigpio, claiming to implement interrupt handling for GPIO signals. But as far as I can estimate, they all do polling on the pins, therefore implement a busy wait in a parallel thread. Only the library bcm2835 warns that it's not using real interrupts.

Reviewing the available datasheets (BCM2835 and BCM2711) I was able to identify the registers on which one can enable interrupts for three pins, which then should be routed from the Video Core over the Generic Interrupt Controller (GIC-400) to the CPU cores. On ATmega32 you can write to registers like these to enable interrupts on certain pins and implement `ISR(desired_vec)` as interrupt handler method.

Now I'd like to know how to implement an ISR on RPi. Is that even possible in user space?

And what is the difference between interrupts and BCM's edge detection? Does this actually activate the GIC?

gpio **c** **interrupts**

Share Improve this question

edited Jan 11, 2021 at 14:08

asked Jan 11, 2021 at 12:13

Follow



void

43

1

5

A good question! – [Seamus](#) Jan 11, 2021 at 18:57

3 Answers

Sorted by: Highest score (default)





3



There is no way to call userspace code from an ISR. Unlike system calls which run on the stack of the userspace program, interrupt handlers use internal kernel memory for the stack. Since that memory is not visible in userspace, the system would crash the moment your ISR userspace function finishes or tries to use the stack for local variables (if not before, due to other reasons I have overlooked).

If you need to play with interrupts, you need to write a kernel driver.

Share Improve this answer Follow

answered Jan 11, 2021 at 15:22



Dmitry Grigoryev

28.2k 6 54 145

Thanks for this clear no. Can you advise me any reference on how to handle this on kernel level? Would I have to write to the device tree like [here](#)? And having this done, I guess I need a bit code that implements my interrupt handling? And actually there was a second question: What exactly is edge detection in comparison? – [void](#) Jan 11, 2021 at 19:09

AFAIK there's no way to define interrupts from userspace either, OFC every OS under the sun has to deal with interrupts but I understand it's something possible at kernel level only. And sure there are provisions to build things looking like an interrupt. – [MaxDZ8](#) Jan 17, 2021 at 15:55

- 1 @MaxDZ8 "sure there are provisions to build things looking like an interrupt" - yes, that would be `poll()` / `select()` and similar functions. Since the kernel cannot call a userspace function, the userspace program has to call the kernel, and returning from the kernel will indicate an event. – [Dmitry Grigoryev](#) Jan 18, 2021 at 8:23

Excuse me @DmitryGrigoryev but I doubt your answers are effective in solving the issue we can observe. There is an ongoing misunderstanding `poll/select` being interrupt-based ARE interrupts. I think the difference is pretty obvious to us. – [MaxDZ8](#) Jan 19, 2021 at 7:20

@MaxDZ8 If by "issue" you mean userspace interrupts, then there is no solution. Userspace interrupts are not possible in Linux. – [Dmitry Grigoryev](#) Jan 19, 2021 at 13:57



wiringPi uses interrupts, e.g. with the wiringPiISR function.

4

pigpio uses interrupts, e.g. with the gpioSetISRFunc function.



lgpio uses interrupts. e.g. with the gGpioSetAlertsFunc function.



None of the above use polling or busy waits. I can only assume you are confused because at a low level they use a Linux function called poll. But this function does not poll the GPIO in the sense you mean.

Linux handles interrupts. As part of its interrupt handling it will eventually schedule one of the above functions.

pigpio can additionally use GPIO polling via DMA which happens to be more accurate and reliable for short (few μ s) level changes.

Share Improve this answer Follow

answered Jan 11, 2021 at 12:28



joan

71.8k

5

75

108

1 Yes, that really confuses me. The manpage on poll says it waits for one of a set of file descriptors to become ready to perform I/O . That's not what I would equalize with an IRQ. I'd expect more like a `raise_softirq()` on a kernel module; Also e.g. the wiringPi code clearly starts an own thread to handle this poll mechanism – void Jan 11, 2021 at 13:31



wiringPi and pigpio DO NOT use interrupts! They create a new thread for each gpio and pool the exported gpio.

0

Share Improve this answer

edited Mar 14, 2024 at 17:14

answered Mar 14, 2024 at 17:14



Follow



Paulo de Almeida

1

1



Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions